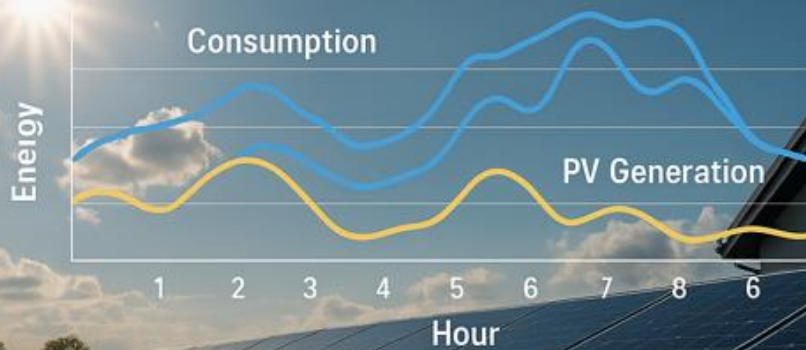


HOURLY ENERGY FORECASTING FOR A PROSUMER



***Previsione del Fabbisogno e Bilancio
Energetico per Prosumer Fotovoltaico***

Sommario

Previsione del Fabbisogno e Bilancio Energetico per Prosumer Fotovoltaico	3
Introduzione	3
Descrizione generale dell'applicazione	3
Perché un modello Encoder-Decoder per serie temporali	4
Componenti principali dell'architettura implementata	4
Logica di addestramento	7
Dettagli e accorgimenti per risultati soddisfacenti	9
Differenze con i modelli Sequential di Keras	10
In conclusione.....	11
Descrizione delle librerie usate	12
Descrizione delle funzioni realizzate	13
Definizione dei Parametri Generali	26
Parametri temporali e di finestra	26
Parametri di suddivisione del dataset	27
Parametri fisici per produzione FV	28
Parametri di consumo energetico	29
Descrizione del flusso del programma	30
Conclusione	35

Previsione del Fabbisogno e Bilancio Energetico per Prosumer Fotovoltaico

Introduzione

Questa applicazione affronta il tema della previsione energetica oraria per un prosumer, ovvero un soggetto che contemporaneamente consuma e produce energia (fotovoltaica). L'obiettivo è **stimare, per ciascuna ora dei prossimi sette giorni, quanto verrà consumato e quanto verrà generato dal sistema FV, in modo da valutare il bilancio energetico e identificare il fabbisogno aggiuntivo da coprire con fonti convenzionali o da immettere in rete**. Anche se il problema richiede tecniche di machine learning avanzate (reti neurali LSTM sequenziali), la documentazione si propone di presentare in modo accessibile i concetti di base, l'architettura del programma e il flusso operativo, lasciando spazio a illustrazioni, grafici e commenti che l'utente inserirà in un secondo momento.

Descrizione generale dell'applicazione

L'applicazione è pensata per simulare in ambiente controllato un flusso completo di forecasting energetico: genera dati sintetici di meteo, consumo e produzione storica, addestra modelli **LSTM seq2seq** ¹ per prevedere consumo e produzione, valuta le performance mediante metriche standard e produce forecast orario per i sette giorni successivi. Il nucleo tecnico utilizza Python con librerie quali NumPy, pandas e TensorFlow/Keras per le reti neurali. Sebbene il contesto sembri complesso, il programma organizza chiaramente ogni fase: dalla preparazione e pulizia dei dati, alla creazione di sequenze per l'encoder-decoder ², alla costruzione dei modelli, fino alla valutazione e alla generazione di report e grafici.

L'applicazione ha finalità doppie: da un lato illustrare uno studio di fattibilità di *forecasting multivariato* ³ in ambito energetico, dall'altro fungere da manuale d'uso del software sviluppato. Chi utilizza il programma potrà sostituire i dati sintetici con dati reali, modificare parametri (ad esempio la capacità FV o la lunghezza delle finestre),

¹ Un modello **Sequence-to-Sequence (Seq2Seq)** è un'architettura neurale progettata per trasformare una sequenza di input in una sequenza di output, potenzialmente di lunghezza diversa. È ampiamente utilizzato in ambiti come la traduzione automatica, il riassunto testuale e la previsione di serie temporali.

² L'architettura **encoder-decoder** è un modello di rete neurale in due fasi in cui l'**encoder** comprime una sequenza di input in una rappresentazione compatta (stato interno), mentre il **decoder** utilizza tale rappresentazione per generare una sequenza di output. È particolarmente utile quando input e output hanno lunghezze diverse o rappresentano concetti diversi, come nella traduzione automatica o nella previsione multistep di serie temporali.

³ Il **forecasting multivariato** in ambito energetico consiste nella previsione di variabili energetiche (es. consumi, produzione da rinnovabili) utilizzando contemporaneamente più serie storiche correlate, come temperatura, irradiazione solare, velocità del vento, orario e dati di consumo/produzione passati. Questo approccio consente di catturare le interazioni tra fattori climatici e comportamentali, migliorando l'accuratezza delle previsioni rispetto ai modelli univariati.

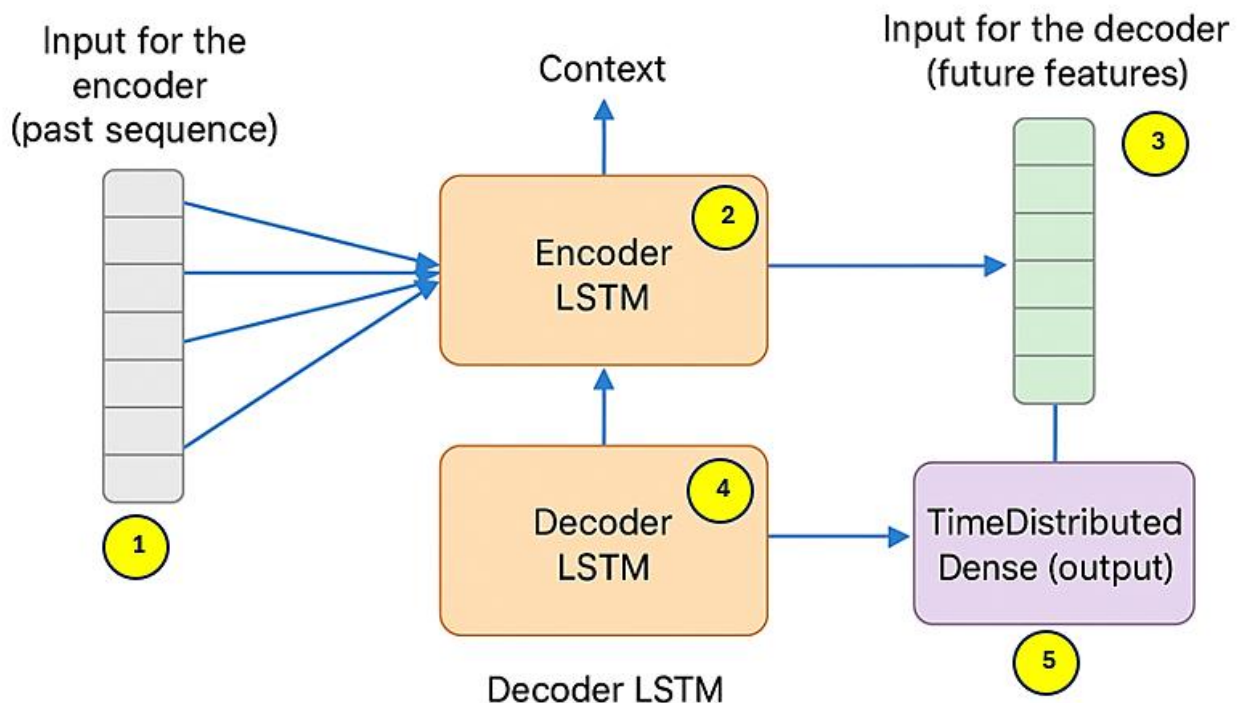
estendere le funzioni o aggiungere nuove variabili esplicative. La documentazione qui proposta fornisce una guida tecnica dettagliata e una panoramica fluida, in linguaggio informatico ma non eccessivamente schematico, così da risultare fruibile sia da chi conosce già il machine learning sia da chi si avvicina per la prima volta a queste tematiche.

Perché un modello Encoder-Decoder per serie temporali

Quando vogliamo prevedere una intera finestra di valori futuri (ad esempio le prossime 168 ore di consumo o produzione) sulla base di una finestra di dati passati e di informazioni esogene (come condizioni meteo future), serve un'architettura che “comprenda” il contesto storico e “utilizzi” i dati esterni futuri per generare una sequenza di output. Il pattern **encoder-decoder**, tipico nella traduzione automatica, si presta bene a questo compito: l'**encoder** “riassume” i dati passati in uno stato interno, e il **decoder** “scompone” quello stato insieme alle informazioni future per produrre la sequenza di previsioni.

Componenti principali dell'architettura implementata

In sintesi l'architettura è così composta.



1. Input per l'encoder (sequenza passata)

- Raccoglie, per ogni ora della finestra storica (e.g. 168 ore), tutte le feature rilevanti:
 - Variabili di contesto/meteo (temperature, irradiance, wind_speed, ecc.)
 - Variabili "target" se includi il passato del target nell'encoder (ad esempio il consumo o la produzione storica). Nel nostro caso, per il modello consumo si ha un dataframe di feature che include anche la serie "*consumption*" passata; per produzione, analogamente includi la serie "*production*" passata insieme al meteo storico.
- Struttura dati: un array tridimensionale di forma (batch_size, past_window, n_features_enc) ⁴. Ogni "finestra" di addestramento è un campione.

2. Encoder LSTM

- È una rete **LSTM** ⁵ che riceve la sequenza delle feature passate.
- Scorre temporalmente le 168 ore storiche: alla fine restituisce gli stati interni finali (*hidden state* e *cell state*) ⁶.
- Quegli stati finali sono un "riassunto numerico" del comportamento recente: contengono informazioni su trend, pattern giornalieri/settimanali, ciclicità, eventuali salti o anomalie.
- Il parametro *latent_dim* (numero di unità LSTM) rappresenta la dimensione dello spazio latente, ovvero il numero di unità (neuroni) della LSTM che definiscono la capacità di memoria e di rappresentazione interna del modello. Un valore maggiore permette al modello di apprendere strutture più complesse nei dati, ma aumenta anche il rischio di overfitting e il tempo di addestramento.

⁴ **batch_size** – Numero di sequenze (o esempi) elaborate insieme durante ogni passo dell'addestramento. Un valore maggiore può velocizzare il training, ma richiede più memoria. **past_window** – Lunghezza temporale della finestra di input, cioè **quante ore passate** vengono fornite all'encoder per generare una previsione (es. 168 ore = 7 giorni). **n_features** – Numero di variabili (feature) per ciascun timestep della sequenza. Può includere, ad esempio, temperatura, irraggiamento, consumo, ecc.

⁵ Le reti **LSTM** (*Long Short-Term Memory*) sono un tipo di rete neurale ricorrente progettata per apprendere dipendenze a lungo termine all'interno di sequenze temporali. Grazie a una struttura interna fatta di "celle di memoria" e "porte" (input, output e forget), le LSTM sono particolarmente efficaci nella modellazione di fenomeni sequenziali come i consumi energetici, riuscendo a ricordare informazioni rilevanti anche dopo molte unità di tempo.

⁶ Nelle reti **LSTM**, il *hidden state* (stato nascosto) rappresenta l'output corrente della rete e contiene informazioni utili alla previsione in corso. Il *cell state* (stato della cella) è una sorta di memoria a lungo termine che attraversa tutta la sequenza, accumulando e trasportando informazioni importanti nel tempo. Insieme, questi due stati permettono alla rete di "ricordare" e "dimenticare" selettivamente ciò che serve per catturare le dipendenze temporali in modo efficace.

3. Input per il decoder (feature future)

- Non si forniscono i target veri come input al decoder (come in alcuni approcci di teacher forcing), ma si utilizzano le feature esogene previste per le ore future: ad esempio la serie di previsioni meteo per le prossime 168 ore.
- Questo array ha forma (*batch_size*, *future_window*, *n_features_dec*), dove **n_features_dec** è tipicamente il numero di variabili meteo (temperature, irradiance, wind_speed). In altre versioni si possono includere anche dati lag dei target, ma nel nostro schema so usano solo informazioni esogene future.
- Lo scopo è dare al decoder “il contesto esterno” ora per ora: ad esempio, sapendo che alle 14:00 domani l’irradiazione sarà alta, il decoder userà quell’informazione per stimare la produzione o l’effetto sul consumo.

4. Decoder LSTM

- Prende in input la sequenza futura di feature, un passo alla volta, e parte dagli stati finali prodotti dall’encoder come “stato iniziale” (**initial_state**).
- In questo modo, ad ogni ora futura, il decoder conosce sia il contesto storico (tramite gli stati) sia il contesto esterno previsto (la feature esogena di quella ora).
- Produce una sequenza di output (uno per ogni passo futuro). Internamente, ogni step del decoder elabora la feature di quell’ora e lo stato ricavato, aggiornando gli stati per il passo successivo.

5. TimeDistributed Dense (strato di output)

- Poiché vogliamo una previsione oraria scalare (ad es. un valore di consumo o produzione per ogni ora), si applica uno **strato Dense** ⁷ a ciascun output del decoder LSTM.
- L’architettura in Keras è un *TimeDistributed(Dense(1))* ⁸, che trasforma la sequenza di hidden vectors LSTM di dimensione *latent_dim* in una sequenza di valori scalari.

⁷ Uno **strato Dense** (o pienamente connesso) è un livello di rete neurale in cui ogni neurone è connesso a tutti quelli dello strato precedente. Nel forecasting, viene spesso usato alla fine del modello per trasformare l’output della rete (es. LSTM) in una previsione finale, come un valore numerico continuo per ogni istante di tempo.

⁸ Il layer **TimeDistributed(Dense(1))** applica uno **strato Dense** (con un solo neurone) in modo indipendente a ciascun passo temporale della sequenza in uscita dal decoder. Serve a generare una **previsione oraria** (o per ogni timestep) trasformando l’output della LSTM in una sequenza di valori scalari, uno per ogni ora prevista.

- Il risultato è un array di forma (*batch_size*, *future_window*, 1), che poi si appiattisce in fase di valutazione o si lascia come sequenza per calcolare la loss.

Logica di addestramento

1. Preparazione dei dati (sliding window)

- Dal dataset storico *df_hist*, si ricavano molte coppie (*X_enc*, *X_dec*, *y*) scorrendo con una finestra mobile:
 - Per ogni possibile posizione temporale *i*, si prende un blocco di *past_window* ore di feature passate come input encoder.
 - Si prende il blocco di *future_window* ore successive di feature esogene (meteo) come input decoder.
 - Si prende la sequenza vera di target (consumo o produzione) di quelle stesse *future_window* ore come *y*.
- Questo genera molti esempi di addestramento, utili per far “vedere” al modello diversi pattern stagionali, diversi momenti della settimana, variazioni meteo, ecc.

2. Divisione train/validation/test

- Il dataset generato viene suddiviso rispettando l'ordine temporale (senza mescolare), con percentuali come 70% train, 15% validation e 15% test.
- Durante l'addestramento, si usa la parte train per ottimizzare i pesi LSTM, e la parte validation per monitorare la loss su dati non visti: se la validation loss smette di migliorare o peggiora, si attiva early stopping per evitare overfitting.
- Infine, su test set si valuta la generalizzazione definitiva dopo l'addestramento.

3. Configurazione del modello

- Viene creato un modello Keras con due Input: uno per l'encoder, uno per il decoder.

- Si compila con loss MSE (mean squared error) ⁹, ottimizzatore Adam ¹⁰ (adattivo, spesso efficace su serie temporali).
- Si scelgono iperparametri: dimensione latent_dim delle LSTM, numero di epoche ¹¹, batch size, learning rate ¹² di Adam eventualmente regolabile.

4. Training vero e proprio

- Si utilizza `model.fit([...], y, validation_split=...)` o direttamente `validation_data`, con `callback` di `EarlyStopping` ¹³ (monitor val_loss, patience) e `ModelCheckpoint` se si vuole salvare il miglior modello.
- Durante ogni epoca, il modello valuta la loss su train e validation. Si osserva la curva di perdita: deve scendere inizialmente su entrambi; se la training loss scende ma validation loss risale, c'è overfitting e conviene fermarsi.
- Spesso si scala o normalizza i dati (ad es. `MinMaxScaler` o `StandardScaler` su ciascuna variabile) ¹⁴ prima di creare le sequence, per stabilizzare il training delle LSTM.

5. Valutazione delle metriche

- Dopo il training, si calcolano predizioni su train e su test:

`y_pred_train = model.predict([X_enc_train, X_dec_train])`,

`y_pred_test = model.predict([X_enc_test, X_dec_test])`.

⁹ **MSE (Mean Squared Error)** – È una metrica di valutazione che misura l'errore quadratico medio tra i valori reali e quelli previsti dal modello. Penalizza maggiormente gli errori grandi, ed è ampiamente usato nei problemi di regressione e nelle reti neurali per serie temporali. Valori più bassi indicano una previsione più accurata.

¹⁰ **Adam (Adaptive Moment Estimation)** – È un ottimizzatore molto usato nelle reti neurali perché combina i vantaggi di due tecniche: **momentum** e **adattamento del tasso di apprendimento**. È veloce, efficiente e richiede poca configurazione. Si adatta bene anche ai dati rumorosi o sparsi, rendendolo ideale per applicazioni complesse come il forecasting energetico.

¹¹ **Epoche (epochs)** – Indicano quante volte l'intero dataset di addestramento viene passato attraverso la rete neurale. Ogni epoca è un ciclo completo di apprendimento: più epoche permettono al modello di apprendere meglio, ma troppe possono causare **overfitting** (cioè imparare troppo dai dati, anche dai rumori).

¹² **Learning rate** – È un parametro che controlla la velocità con cui il modello aggiorna i suoi pesi durante l'addestramento. Un valore troppo alto può causare instabilità o mancata convergenza, mentre uno troppo basso può rallentare l'apprendimento o bloccarlo in soluzioni subottimali. È uno dei parametri più delicati da impostare.

¹³ **EarlyStopping** – È una tecnica di **callback** usata durante l'addestramento per interrompere automaticamente il processo se il modello **smette di migliorare** (ad esempio nella perdita di validazione). Serve a **evitare l'overfitting** e a risparmiare tempo, fermando l'addestramento quando non ci sono più progressi significativi.

¹⁴ **MinMaxScaler**, trasforma i dati scalando ogni feature in un intervallo predefinito (tipicamente [0, 1]), mantenendo la distribuzione originale ma ridimensionando i valori in modo che il minimo diventi 0 e il massimo 1. **StandardScaler**, Standardizza le feature rimuovendo la media e scalando alla varianza unitaria, cioè trasforma i dati in modo che abbiano media 0 e deviazione standard 1, favorendo modelli che assumono distribuzioni gaussiane.

- Si usano funzioni come MAE ¹⁵ e RMSE per misurare l'errore medio e la penalizzazione di errori più grandi. Un valore basso indica che il modello ha appreso bene i pattern.
- Si verificano grafici reali vs predetti su finestre specifiche, distribuzione degli errori, curve di perdita epoca per epoca.

6. Forecast futuro

- Con il modello addestrato, per prevedere i prossimi 7 giorni si costruisce:
 - `X_enc_future` = ultime `past_window` ore di dati storici reali
 - `X_dec_future` = serie di feature meteo previste per le prossime `future_window` ore
- Si chiama `model.predict([X_enc_future, X_dec_future])` per ottenere la sequenza di previsioni.
- Questo approccio garantisce che il modello utilizzi la memoria del contesto passato e le informazioni esogene future per produrre una previsione coerente oraria.

Dettagli e accorgimenti per risultati soddisfacenti

- **Scaling dei dati:** Normalizzare le feature (temperatura, irradianza, consumo, produzione) su range simili aiuta le LSTM a convergere meglio. Solitamente si applica uno scaler (*MinMax* o *Standard*) basato sui valori storici, poi si trasforma train/val/test. Le previsioni future dovrebbero essere scalate con lo stesso scaler.
- **Iperparametri LSTM:** La dimensione dello stato interno (*latent_dim*) non deve essere né troppo piccola (rischio sottofit) né troppo grande (overfit). Si sperimenta con valori come 32, 64, 128.
- **Numero di epoche e batch size:** Occorre osservare le curve di loss: un batch size di 32 o 64 è comune; le epoche si fermano con EarlyStopping non appena la validation loss non migliora più.
- **Regularizzazione:** Se noti overfitting, puoi aggiungere dropout o recurrent dropout all'interno delle LSTM, o L2 sui pesi.
- **Feature engineering:** Oltre al meteo, potrebbe aiutare includere variabili temporali (ora del giorno, giorno della settimana, festivi) come feature aggiuntive sia nell'encoder che, se prevedibili, nel decoder.

¹⁵ **MAE (Mean Absolute Error).** È la media degli errori assoluti tra valori previsti e valori reali, misura l'accuratezza delle previsioni indicando quanto, in media, le stime si discostano dai dati osservati.

- **Qualità del meteo futuro:** Il modello assume di avere valori di meteo previsti: nella simulazione si genera meteo sintetico, ma in un contesto reale servirebbe una fonte esterna di previsioni meteo. La qualità del forecast finale dipende anche dall'accuratezza di questi input esogeni.
- **Dimensione del dataset:** Con dati reali, è utile avere diverse stagioni, più anni di storico, per catturare pattern complessi e festività.
- **Verifica della stazionarietà:** Anche se LSTM gestisce trend e stagionalità, in alcuni casi può aiutare differenziare le serie, oppure includere componenti cicliche esplicite.
- **Batching e shuffled windows:** Nel nostro caso si usano finestre temporali contigue senza shuffling all'interno del train (ma Keras di default mescola i batch): è importante mantenere la sequenza storica per split in train/val/test, ma durante l'ottimizzazione i campioni possono essere mescolati, migliorando la robustezza.
- **Monitoraggio dei residui:** Dopo l'addestramento, esaminare il residuo (reale – predetto) per pattern non catturati, errori sistematici in certe ore o condizioni (ad es. sempre sottostima la sera). Questo può suggerire miglioramenti (feature aggiuntive, architettura diversa, ensembling).
- **Verifica su test "indipendente":** Oltre alla parte test ricavata dallo storico, se possibile riservare periodi del passato (es. un anno intero) esclusi dall'addestramento iniziale per una prova più robusta.
- **Riproducibilità:** Fissare seed casuali (numpy, TensorFlow) aiuta a riprodurre i risultati quando si fa tuning.

Differenze con i modelli Sequential di Keras

Ecco una spiegazione sintetica delle differenze principali:

- **Obiettivo multi-step vs one-step**
 - Con un **modello Sequential()** LSTM si tende spesso a prevedere un solo passo avanti (one-step) o, ricorrendo a tecniche iterative, a ripetere la previsione per più passi (recursive forecasting). Questo può accumulare errori a ogni iterazione.
 - Nell'architettura **encoder-decoder seq2seq**, il modello è pensato fin dall'inizio per generare direttamente una sequenza di valori futuri di lunghezza prefissata (multi-step) in un unico passaggio, riducendo la propagazione di errore iterativo.

- **Gestione di feature esogene future**
 - In un *modello Sequential()*, incorporare valori futuri di variabili esterne (es. previsioni meteo) non è immediato: servirebbe concatenare manualmente dati passati e futuri o fare complicate manipolazioni.
 - Nell'*encoder-decoder*, il decoder riceve naturalmente, passo dopo passo, le feature esogene future come input, un meccanismo integrato che aiuta a sfruttare l'informazione nota sul futuro durante la previsione.
- **Separazione contesto storico vs generazione futura**
 - Nel *Sequential()*, l'intera sequenza di input (storico e magari qualche dato futuro) attraversa gli stessi layer senza distinzione esplicita fra "memoria del passato" e "uso delle variabili esterne future".
 - Nell'*encoder-decoder*, l'encoder si occupa di sintetizzare il contesto storico in uno stato interno, mentre il decoder usa questo stato insieme alle feature future, mantenendo chiaramente separati i ruoli di comprensione del passato e generazione del futuro.
- **Flessibilità e complessità**
 - Il *modello Sequential()* è più semplice da definire e sufficiente se serve una previsione one-step o un multi-step molto breve, senza esigenze di feature future complesse.
 - L'*encoder-decoder* è leggermente più complesso da implementare, ma offre maggiore flessibilità per forecasting multistep con input esogeni e gestisce meglio la generazione di intere sequenze di output.

Quindi, se serve prevedere una sequenza di valori futuri in base a un lungo storico e a variabili esogene note per il futuro, l'architettura encoder-decoder è più adeguata; per previsioni un passo alla volta o scenari più semplici, un LSTM in *Sequential()* può bastare e risulta più immediato.

In conclusione

L'architettura **seq2seq** con encoder e decoder LSTM è costruita per "leggere" una sequenza storica di feature (meteo + target passato) e "sfruttare" le features esogene future (meteo) per generare una sequenza di previsioni orarie. Durante l'addestramento si creano molte coppie di finestre passate e future da dati storici; si usano tecniche di normalizzazione, early stopping e validazione per evitare overfitting.

I principali componenti sono:

1. **Encoder LSTM**: sintetizza in uno stato interno le informazioni storiche.
2. **Decoder LSTM**: riceve lo stato interno e le feature future passo per passo, producendo la serie di output.
3. **TimeDistributed Dense**: converte l'output del decoder in valori scalari orari.
4. **Sliding window**: genera campioni per training, validation e test, catturando pattern di lungo periodo.
5. **Callback e monitoraggio**: early stopping su validation loss, salvataggio del miglior modello.
6. **Valutazione**: metriche MAE/RMSE su train e test; grafici perdita e confronto reale vs predetto.

Seguendo questi passaggi con cura (scaling coerente, scelta di iperparametri, qualità degli input meteo, analisi dei residui) si può ottenere un sistema di previsione accurato e utile per pianificare il fabbisogno energetico di un prosumer.

Descrizione delle librerie usate

Nel programma sono importate principalmente:

- **NumPy** e **pandas**: per la gestione di array numerici e serie temporali, creazione di DataFrame e operazioni di slicing, concatenazione, interpolazione e calcolo di statistiche.
- **matplotlib** (e talvolta **seaborn** per istogrammi): impiegate per tracciare grafici di serie storiche, curve di perdita durante l'addestramento, confronti tra reale e predetto, distribuzione degli errori e visualizzazioni di bilancio energetico.
- **scikit-learn** (`mean_absolute_error`, `mean_squared_error`): per calcolare metriche di accuratezza del modello come MAE e RMSE.
- **TensorFlow/Keras**: per definire e addestrare modelli di rete neurale LSTM in architettura seq2seq; in particolare Input, LSTM, Dense, TimeDistributed e callback come EarlyStopping o ModelCheckpoint per il training.
- **datetime**, **os**: per gestione di timestamp e operazioni di sistema, se necessario.
- In alcune versioni compare anche **seaborn** per plottare distribuzioni con funzione `histplot`, ma l'uso principale rimane matplotlib.

Queste librerie formano la base di un moderno workflow in Python per data science e deep learning, offrendo funzionalità di preprocessing, modellazione sequenziale e visualizzazione, consentendo di costruire un prototipo di forecasting energetico robusto e modulare.

Descrizione delle funzioni realizzate

Il programma definisce varie funzioni modulari, ognuna con uno scopo preciso:

- **genera_date_range_storico(months, freq, tz)**
 - Obiettivo/Scopo: restituire un indice temporale orario (DatetimeIndex) che copre gli ultimi “months” mesi fino al momento corrente.
 - Descrizione: calcola end come ora corrente arrotondata all’ora in fuso Europe/Rome, sottrae il periodo in mesi e genera una serie di timestamp con frequenza oraria. Serve a definire l’orizzonte storico da cui partire per la simulazione.
- **genera_meteo_sintetico(index)**
 - Obiettivo/Scopo: simulare dati meteo orari (temperatura, irradiance, wind_speed) su un indice temporale dato.
 - Descrizione: combina pattern sinusoidali giornalieri e stagionali per la temperatura, una sinusoide che modella l’irradianza (zero di notte, picco a mezzogiorno) con rumore gaussiano e una produzione di vento media con variabilità casuale. Restituisce DataFrame con colonne appropriate e indice orario. Le immagini sotto è un tipico esempio. La prima è riferita ad un periodo nel passato, la seconda ai prossimi 7 giorni.

Generazione dataset meteo...

	temperature	irradiance	wind_speed
2024-12-22 08:00:00+01:00	29.368270	0.847078	3.811933
2024-12-22 09:00:00+01:00	23.053792	0.922625	4.673892
2024-12-22 10:00:00+01:00	23.455223	0.956161	4.005090
2024-12-22 11:00:00+01:00	18.973638	0.980126	1.762669
2024-12-22 12:00:00+01:00	15.023488	0.964870	3.426071

Generazione previsioni meteo sintetiche per i prossimi 7 giorni...

	temperature	irradiance	wind_speed
2025-06-22 09:00:00+02:00	30.554315	0.808179	4.699946
2025-06-22 10:00:00+02:00	27.973796	0.958285	3.490483
2025-06-22 11:00:00+02:00	25.594080	0.910835	3.758339
2025-06-22 12:00:00+02:00	20.322373	0.981048	3.262027
2025-06-22 13:00:00+02:00	18.723028	1.000000	4.015274

- **genera_consumo_sintetico(index, meteo_df)**

- Obiettivo/Scopo: simulare il profilo orario di consumo energetico in funzione dell'orario della giornata e della temperatura esterna.
- Descrizione: calcola un pattern giornaliero (picchi mattina/sera), aggiunge un effetto temperatura se la temperatura supera soglie (ad esempio $>22^{\circ}\text{C}$ o $<16^{\circ}\text{C}$) con un fattore empirico, include rumore casuale e fa clipping per non avere valori negativi. Restituisce Series "consumption". L'immagine sotto è un tipico esempio.

```
Generazione dataset Consumo...
2024-12-22 08:00:00+01:00    1.547900
2024-12-22 09:00:00+01:00    1.367088
2024-12-22 10:00:00+01:00    1.406859
2024-12-22 11:00:00+01:00    1.288799
2024-12-22 12:00:00+01:00    1.546909
Freq: h, Name: consumption, dtype: float64
```

- **genera_produzione_sintetica(index, meteo_df)**

- Obiettivo/Scopo: simulare la produzione oraria di un impianto fotovoltaico basata sull'irradiazione.
- Descrizione: utilizza PANEL_CAPACITY e EFFICIENCY come parametri fisici, moltiplica per il valore di irradianza per ora, aggiunge rumore, effettua clipping. Restituisce Series "production". L'immagine sotto è un tipico esempio.

```
2024-12-22 08:00:00+01:00    0.819483
2024-12-22 09:00:00+01:00    0.841816
2024-12-22 10:00:00+01:00    0.862677
2024-12-22 11:00:00+01:00    0.924551
2024-12-22 12:00:00+01:00    0.801658
Freq: h, Name: production, dtype: float64
```

- **pulisci_e_imputa(df)**

- Obiettivo/Scopo: introdurre valori mancanti casuali e poi ripulire con interpolazione temporale e forward/backfill.
- Descrizione: simula buchi nei dati, quindi ricostruisce la serie senza NaN con metodi di interpolazione di pandas, garantendo un dataset completo per il training.

- **split_time_series(df, train_ratio, val_ratio)**

- Obiettivo/Scopo: suddividere in modo sequenziale (senza shuffling) un DataFrame temporale in train, validation e test set secondo percentuali date.

- Descrizione: calcola il numero di record per ogni porzione, restituisce tre DataFrame contigui. L'immagine di sotto ci fornisce le indicazioni rilasciate. La lunghezza dell'intero dataset iniziale e le lunghezze dei dataset di Train, Val e Test derivate dai rapporti di ripartizione. Inoltre ci fornisce indicazione del periodo di inizio e fine di ognuno di essi.

```
Lunghezza dataset : 4368
Lunghezza Train : 3057
Lunghezza Val : 655
Lunghezza Test : 656
-----
Suddivisione in train/val/test...
TRAIN: da 2024-12-22 08:00:00+01:00 a 2025-04-28 17:00:00+02:00
VAL: da 2025-04-28 18:00:00+02:00 a 2025-05-26 00:00:00+02:00
TEST: da 2025-05-26 01:00:00+02:00 a 2025-06-22 08:00:00+02:00
```

- **analisi_statistica(df, title_prefix)**

- Obiettivo/Scopo: condurre un'analisi esplorativa rapida su un dataset (train/val/test): statistiche descrittive, grafico delle serie, matrice di correlazione con heatmap.
- Descrizione: stampa i **valori statistici dell'analisi** df.describe(), traccia subplots di ogni colonna, calcola la **mappa di correlazione** df.corr(), e disegna la **conseguente heatmap**. Aiuta a comprendere relazioni tra variabili e individuare anomalie o pattern. La prima immagine di sotto ci fornisce i dati statistici dell'analisi condotta sui valori di temperatura, irradiazione, velocità del vento, consumo e produzione da FV.

```
Analisi statistica TRAIN...
Statistiche descrittive TRAIN:
```

	temperature	irradiance	wind_speed	consumption	production
count	3057.000000	3057.000000	3057.000000	3057.000000	3057.000000
mean	29.845897	0.635308	2.986646	1.590801	0.572353
std	9.136983	0.309177	0.990180	0.406247	0.292110
min	2.210410	0.000000	0.000000	0.399870	0.000000
25%	23.574393	0.383124	2.310737	1.301128	0.342801
50%	29.656880	0.703523	3.008731	1.582137	0.629661
75%	37.317710	0.915777	3.660816	1.884131	0.809913
max	49.476227	1.000000	6.132836	2.681816	1.204631

In particolare :

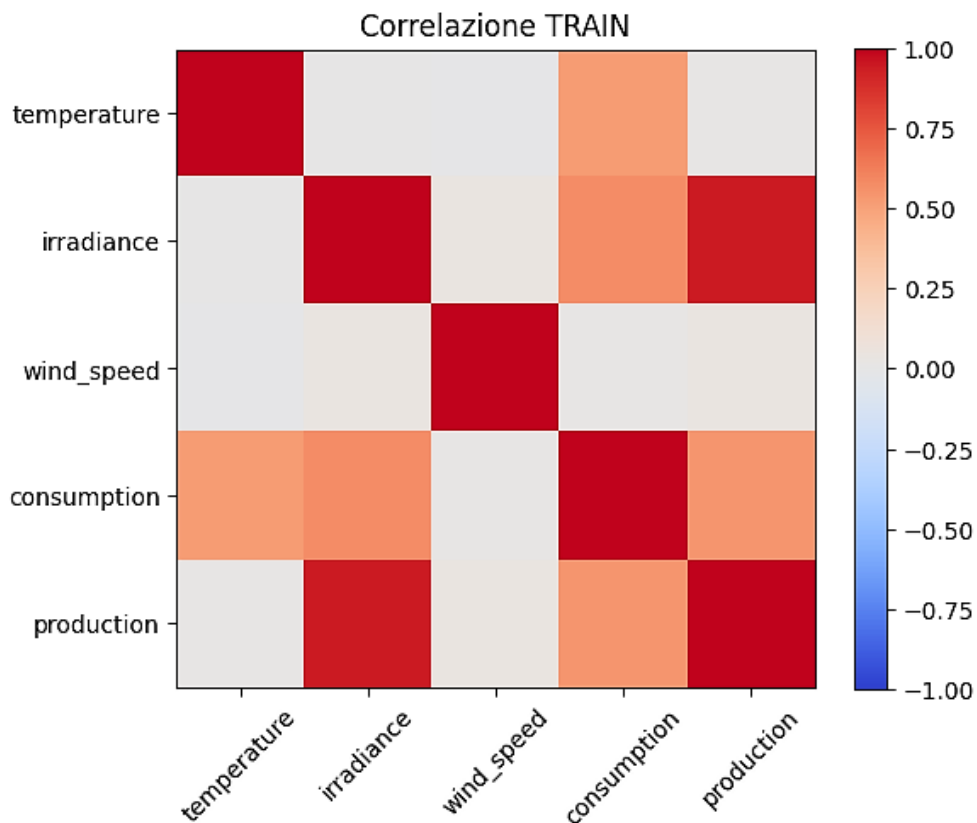
- **Count:** numero totale di osservazioni non mancanti.
- **Mean:** media aritmetica dei valori (valore medio).
- **Std:** deviazione standard, misura la dispersione dei dati rispetto alla media.
- **Min, Max:** valori minimo e massimo osservati nella distribuzione.

- **25%, 50%, 75%:** percentili; indicano i valori sotto i quali si trova rispettivamente il 25%, 50% (mediana) e 75% dei dati ordinati. Aiutano a comprendere la distribuzione dei dati.

Poi fornisce la **Matrice di correlazione** delle caratteristiche sui dataset di TRAIN, VAL e TEST...

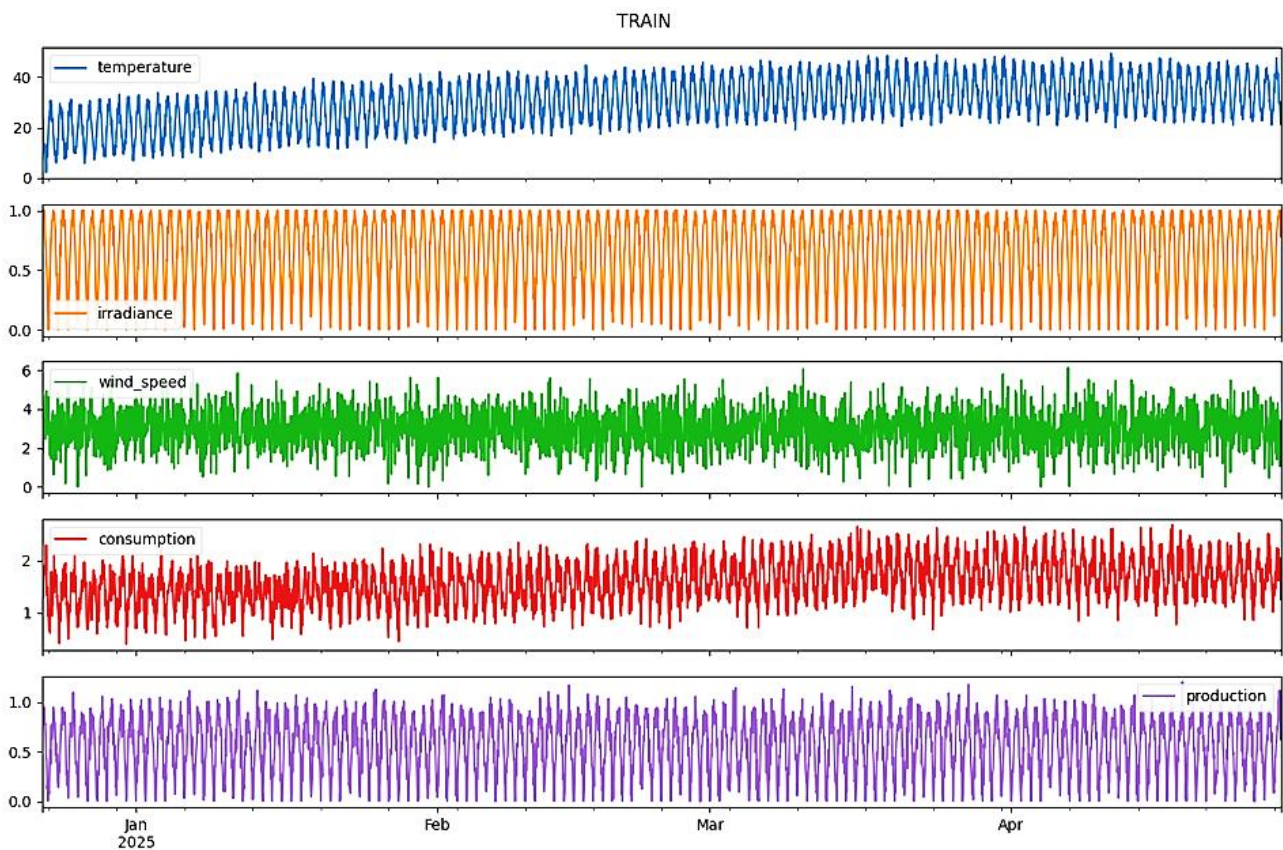
Matrice di correlazione TRAIN:					
	temperature	irradiance	wind_speed	consumption	production
temperature	1.000000	0.001042	-0.014989	0.520431	0.010671
irradiance	0.001042	1.000000	0.038752	0.570995	0.940129
wind_speed	-0.014989	0.038752	1.000000	0.015166	0.036405
consumption	0.520431	0.570995	0.015166	1.000000	0.540432
production	0.010671	0.940129	0.036405	0.540432	1.000000

... e la relativa **Heatmap**...



Infine fornisce un grafico con l'andamento nel tempo (analizzato nel periodo di analisi) dei valori di :

- Temperatura
- Irradiazione
- Velocità del vento
- Consumo
- Produzione da Fotovoltaico



- **crea_sequence_dataset(df_features_past, df_targets_past, df_features_future, df_targets_future, past_window, future_window)**
 - Obiettivo/Scopo: preparare i dati per l'addestramento di un modello seq2seq LSTM, creando sequenze fisse di input per encoder e decoder e i target corrispondenti.
 - Descrizione: scorre con sliding window sul dataset storico: per ciascun possibile punto i , prende `past_window` ore di feature passate come input encoder, `future_window` ore di feature future (decoder) e i valori futuri di target per la loss. Restituisce array NumPy tridimensionali per encoder e decoder e matrice target.
- **build_seq2seq_model(n_features_enc, n_features_dec, latent_dim, future_window)**
 - Obiettivo/Scopo: definire in Keras un modello LSTM seq2seq per forecasting multistep.
 - Descrizione: crea Input per encoder (sequenza passata), un LSTM che restituisce stati (h , c), Input per decoder (feature future), un altro LSTM che utilizza gli stati dell'encoder, seguito da un `TimeDistributed(Dense(1))`

per restituire la sequenza di output di lunghezza future_window. Compila con optimizer Adam e loss MSE. Una volta istanziata la funzione il programma prima costruirà il modello Seq2Seq, poi provvederà all'addestramento (avideo comparirà un monitor che evidenzia le fasi di addestramento per le varie Epochs previste...

```
Costruzione modello seq2seq... (esempio generico)
-----
Modello dei Consumi
***** ADDESTRAMENTO MODELLO CONSUMI *****
Epoch 1/30
73/73 [=====] - 29s 286ms/step - loss: 0.3951 - val_loss: 0.0959
Epoch 2/30
73/73 [=====] - 19s 256ms/step - loss: 0.0786 - val_loss: 0.0526
Epoch 3/30
73/73 [=====] - 19s 262ms/step - loss: 0.0488 - val_loss: 0.0463
Epoch 4/30
73/73 [=====] - 19s 257ms/step - loss: 0.0456 - val_loss: 0.0458
Epoch 5/30
73/73 [=====] - 19s 256ms/step - loss: 0.0445 - val_loss: 0.0454
```

ed infine visualizzerà le caratteristiche del modello utilizzato sia sul dataset consumi, sia su quello della produzione da FV.

Layer (type)	Output Shape	Param #	Connected to
encoder_inputs (InputLayer)	[(None, None, 4)]	0	[]
decoder_inputs (InputLayer)	[(None, None, 3)]	0	[]
encoder_lstm (LSTM)	[(None, 64), (None, 64), (None, 64)]	17664	['encoder_inputs[0][0]']
decoder_lstm (LSTM)	[(None, None, 64), (None, 64), (None, 64)]	17408	['decoder_inputs[0][0]', 'encoder_lstm[0][1]', 'encoder_lstm[0][2]']
decoder_dense (TimeDistributed)	(None, None, 1)	65	['decoder_lstm[0][0]']
=====			
Total params: 35137 (137.25 KB)			
Trainable params: 35137 (137.25 KB)			
Non-trainable params: 0 (0.00 Byte)			

Come si può notare, per addestrare il modello descritto nell'immagine sul dataset dei consumi sono stati utilizzati oltre 3500 parametri.

- **valuta_metriche(y_true, y_pred)**

- Obiettivo/Scopo: calcolare metriche di accuratezza (MAE, RMSE) tra valori reali e predetti.
- Descrizione: appiattisce gli array e utilizza funzioni di scikit-learn per MAE e MSE, quindi scompone MSE in RMSE. Restituisce due valori numerici utili al confronto tra diversi modelli o set di dati (train/test).
- L'immagine di sotto ...

```
86/86 [=====] - 10s 93ms/step
✅ Metriche modello Consumo: MAE = 0.1523, RMSE = 0.1905
86/86 [=====] - 10s 91ms/step
✅ Metriche modello Produzione: MAE = 0.0785, RMSE = 0.0984
11/11 [=====] - 1s 86ms/step
✅ Test Consumo: MAE = 0.3524, RMSE = 0.4351
11/11 [=====] - 1s 84ms/step
✅ Test Produzione: MAE = 0.3876, RMSE = 0.4476
```

- Ci evidenzia i seguenti valori:
 - ✅ Train modello Consumo: MAE = 0.1523, RMSE = 0.1905
 - ✅ Train modello Produzione: MAE = 0.0785, RMSE = 0.0984
 - ✅ Test Consumo: MAE = 0.3524, RMSE = 0.4351
 - ✅ Test Produzione: MAE = 0.3876, RMSE = 0.4476

Ho sottoposto a ChatGpt l'analisi della valutazione dei dati rilasciati e questa è stata la sua risposta:

- Hai ottenuto risultati piuttosto interessanti, ecco un'analisi sintetica ma mirata:
- **✅ Prestazioni sul Training set**
 - **Consumo**
MAE = 0.1523, RMSE = 0.1905
 - Buona accuratezza, con errori medi abbastanza contenuti. RMSE leggermente più alto del MAE indica la presenza di qualche errore più marcato, ma niente di critico.
 - **Produzione**
MAE = 0.0785, RMSE = 0.0984
 - Ottima performance: il modello riesce a catturare molto bene i pattern della produzione nel training, con errori medi bassissimi.

- **Prestazioni sul Test set**

- **Consumo**

- MAE = 0.3524, RMSE = 0.4351

- C'è un aumento significativo degli errori rispetto al training. Il modello generalizza meno bene sui consumi: potresti avere **overfitting**, o il test set presenta variabilità non presente nel training.

- **Produzione**

- MAE = 0.3876, RMSE = 0.4476

- Anche qui l'errore cresce molto rispetto al training, nonostante la produzione fosse ben modellata sul training. Il peggioramento può indicare **sensibilità alle variazioni esterne**, ad esempio cambiamenti climatici o stagionalità non catturata bene.

- **Suggerimenti**

- Il divario tra train e test è evidente, soprattutto per la Produzione: valuta **l'aggiunta di feature esterne** (es. meteo) o l'uso di tecniche di regolarizzazione per migliorare la generalizzazione.

- Controlla se il test set rappresenta stagioni o comportamenti fuori distribuzione rispetto al training.

- Un early stopping ben impostato o una cross-validation a blocchi temporali può aiutare a evitare overfitting su serie temporali.

- **Cosa si può fare per migliorare le metriche sul test set**

- 1. **Regularizzazione** - Applica tecniche per evitare l'overfitting:

- Dropout (se usi reti neurali): aggiungi dropout nei layer decoder e/o tra encoder-decoder.

- 2. **Riduci la complessità del modello** - Se il modello è troppo grande, può adattarsi troppo bene ai dati di training e non generalizzare:

- Riduci il numero di layer o neuroni (encoder/decoder).
 - Usa meno parametri se la quantità di dati è limitata.

- 3. **Data augmentation per serie temporali** - Per far sì che il modello veda più varietà durante l'addestramento, puoi:

- Aggiungere rumore gaussiano ai dati di input.
 - Usare finestre mobili (rolling window) con overlap maggiore.

5. Feature aggiuntive (soprattutto per la produzione) - Se la produzione è influenzata da condizioni esterne (es. meteo, irraggiamento, giorno/festivo), considera:

- Inserire feature esterne esplicite nel decoder input.
- Codificare la stagionalità (es. month, day_of_year, hour come variabili numeriche o one-hot).

6. Ribilanciamento del training/test - Verifica se c'è uno shift di distribuzione tra train e test:

- Le serie coprono periodi differenti? (es. stagioni diverse?)
- Se sì, valuta una normalizzazione dinamica (MinMaxScaler calcolato su rolling window o per mese).

8. Rituning degli iperparametri - Piccole variazioni di:

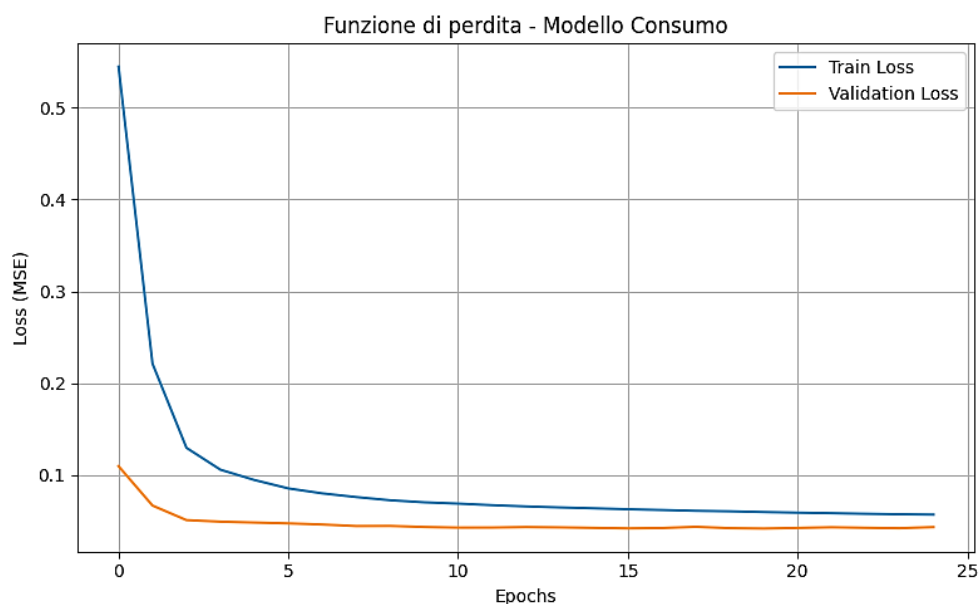
- learning_rate
- batch_size
- past_window e forecast_horizon

possono fare una differenza significativa.

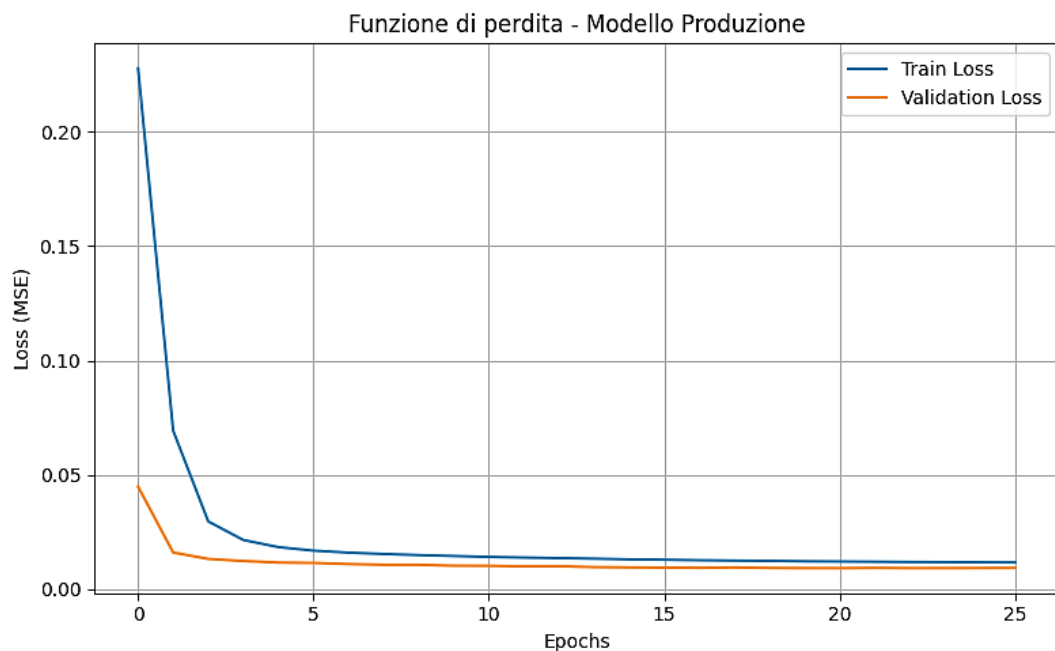
NB: Questi suggerimenti evidenziano quanto sia importante rivedere continuamente il modello adottato. È un processo delicato, che richiede pazienza e perseveranza per raggiungere risultati soddisfacenti. Non esistono framework universali in grado di garantire il successo: si procede per tentativi, affinando il modello passo dopo passo.

- **plot_loss(history, title)**

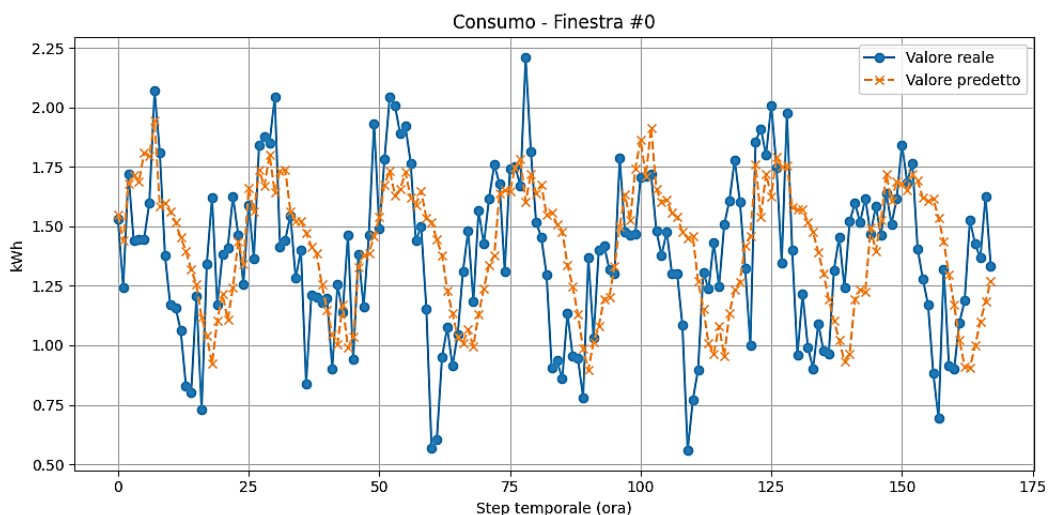
- Obiettivo/Scopo: mostrare l'andamento della funzione di perdita (loss) sui dataset Train e Val durante l'addestramento sul modello di consumo...

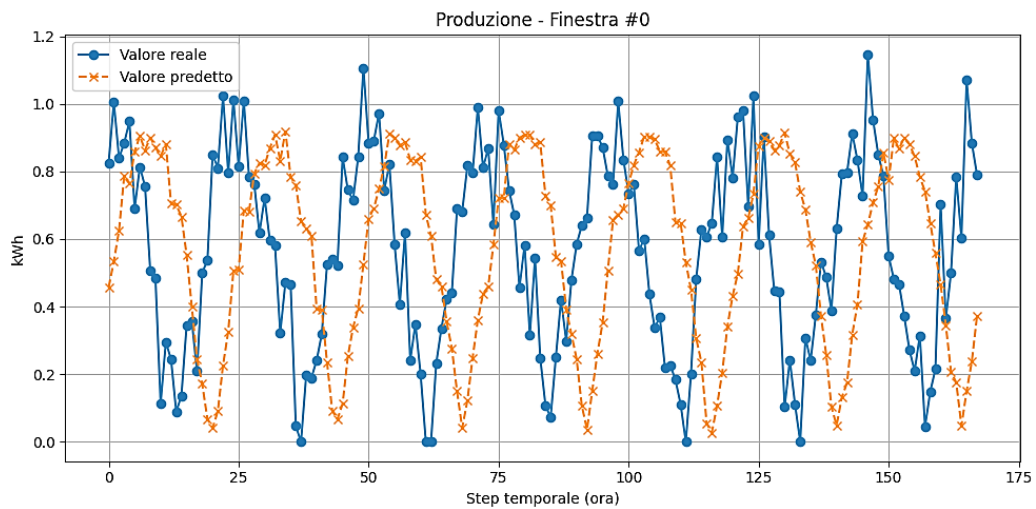


... e su quello di Produzione da fotovoltaico.

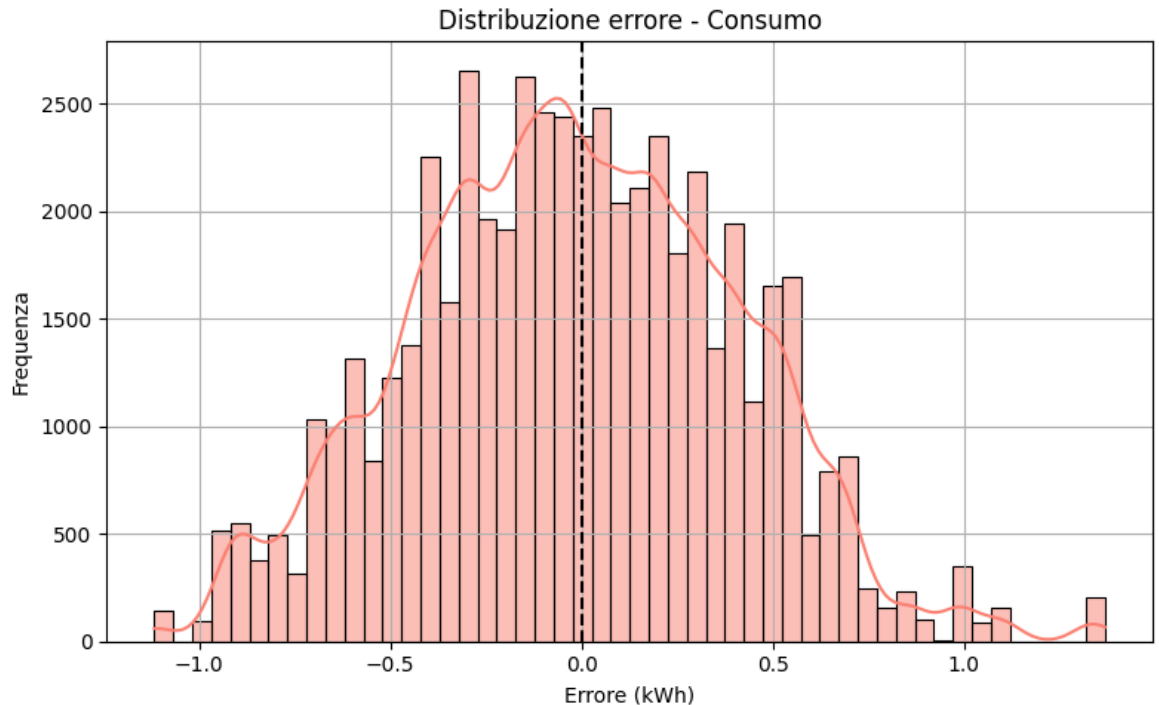


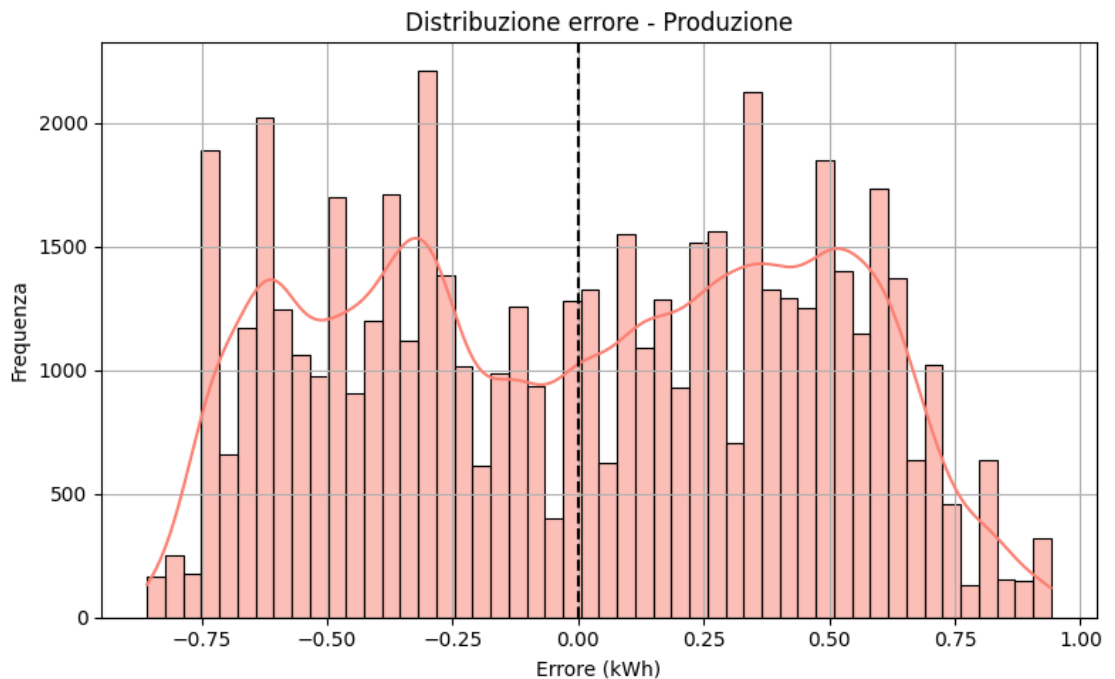
- Descrizione: dati gli oggetti History restituiti da `.fit()`, traccia curve su epoche, evidenziando potenziali overfitting o convergenza.
- Analizzando le due curve si vede chiaramente che l'addestramento ha ottenuto un ottimo risultato (sul modello di Produzione anche migliore di quello di Consumo)
- **`plot_confronto_window(y_true, y_pred, window_idx, title_prefix)`**
 - Obiettivo/Scopo: visualizzare il **confronto reale vs predetto** su una singola finestra temporale (es. 168 ore) per il Consumo e la Produzione.
 - Descrizione: seleziona la finestra di indice `window_idx` (nel nostro caso `window_idx=0`, cioè la prima finestra), estrae i valori reali e predetti, traccia in un grafico a linee con marker differenti per facilitare l'interpretazione puntuale ora per ora.





- **plot_error_distribution(y_true, y_pred, title, bins)**
 - Obiettivo/Scopo: illustrare la distribuzione degli errori (reali – predetti) su tutto il test set.
 - Descrizione: appiattisce gli errori, costruisce un istogramma con KDE su matplotlib/seaborn, aiuta a comprendere bias o asimmetrie nell'errore complessivo.
 - I grafici rilascia sono i seguenti.





✓ Descrizione tipo di grafico:

- È un istogramma che mostra la frequenza degli errori commessi dal modello nel predire i valori rispetto ai dati reali. È molto utile per capire quanto e come il modello sbaglia.
- L'errore per ciascun punto è calcolato come:
 - **errore=valore reale-valore predetto**
- Quindi:
 - **Errore = 0**: il modello ha predetto esattamente il valore reale.
 - **Errore > 0 (a sinistra dello 0)**: il modello ha sottostimato il valore reale → **ha previsto meno di quanto accaduto**.
 - **Errore < 0 (a destra dello 0)**: il modello ha sovrastimato il valore reale → **ha previsto più di quanto accaduto**.
- L'asse X mostra l'intervallo di errore in unità di misura (nel tuo caso, kWh).
- Esempi:
 - -2 kWh: il modello ha predetto 2 kWh in più rispetto al reale.
 - +3 kWh: il modello ha predetto 3 kWh in meno rispetto al reale.
- L'asse Y mostra quante volte (cioè con che frequenza) si è verificato un certo errore. In altre parole, quanti esempi rientrano in ciascun intervallo di errore.
- linea spezzata sopra l'istogramma è una **KDE (Kernel Density Estimate)**, cioè una stima della densità di probabilità degli errori. Aiuta a capire la forma della distribuzione.

- Funziona così:
 - **Se è simmetrica** → errori bilanciati.
 - **Se è spostata a sinistra/destra** → tendenza del modello a sottostimare o sovrastimare.
 - **Se è "larga"** → gli errori sono molto variabili (scarsa precisione).
 - **Se è "stretta e centrata su "0"** → il modello è preciso e accurato.
- La linea verticale nera tratteggiata su $x = 0$ è un riferimento che rappresenta l'errore nullo. Serve per valutare visivamente:
 - quanto la distribuzione degli errori sia centrata sullo zero,
 - se il modello tende a sbagliare sistematicamente (bias).

✅ In sintesi, il grafico serve a capire:

- Quanto spesso il modello sbaglia.
- Se tende a sottostimare o sovrastimare.
- Se gli errori sono distribuiti simmetricamente.
- Quanto sono gravi o contenuti gli errori

Definizione dei Parametri Generali

Parametri temporali e di finestra

Per costruire il dataset storico simulato e definire le sequenze per il modello, impostiamo `HISTORICAL_MONTHS = 6`, `FREQ = 'h'`, `FUTURE_DAYS = 7`, `PAST_WINDOW = 168`, `FUTURE_WINDOW = 168`. In questo modo copriamo sei mesi di storico con passo orario, addestriamo il modello con finestre di sette giorni passati e prevediamo sette giorni futuri, un orizzonte operativo utile per la pianificazione energetica.

- **HISTORICAL_MONTHS = 6**

- **Cos'è:** il numero di mesi di storico che si vogliono considerare per generare i dati sintetici iniziali.
- **A cosa serve:** definisce l'orizzonte temporale passato su cui basare la simulazione e l'addestramento dei modelli. In pratica, serve a creare `idx_hist` (`DatetimeIndex`) con le date e le ore degli ultimi 6 mesi.
- **Dove viene usato:** nella funzione `genera_date_range_storico` (`months=HISTORICAL_MONTHS, ...`) per calcolare il punto di inizio (`start = now - 6 mesi`) e generare la serie oraria storica. Influisce sulla quantità di dati disponibili per l'analisi e l'addestramento: più mesi includi, più pattern stagionali e cicli settimanali il modello potrà apprendere, a costo di un dataset più grande.

- **FREQ = 'h'**

- **Cos'è:** la frequenza temporale utilizzata per costruire gli indici orari.
- **A cosa serve:** indica che si lavora con dati orari ("h" = hourly). Garantisce che tutte le serie (meteo, consumo, produzione) siano su base oraria.
- **Dove viene usato:** in `pd.date_range(..., freq=FREQ)` sia per lo storico (`idx_hist`) sia per le previsioni future (`idx_future`). È essenziale perché modelli LSTM e sequenze sono costruiti su intervalli regolari: uno step corrisponde a un'ora.

- **FUTURE_DAYS = 7**

- **Cos'è:** il numero di giorni futuri da prevedere.
- **A cosa serve:** determina l'orizzonte di forecast: nel programma si prevede l'energia oraria per i prossimi 7 giorni (168 ore).
- **Dove viene usato:** per calcolare `periods = FUTURE_DAYS * 24` in `pd.date_range(start=ultimo_storico + 1 ora, periods=168, freq='h')`, e per dimensionare la finestra del decoder: `future_window = 168`. Scegliere 7 giorni è un compromesso: orizzonte operativo utile per pianificare

approvvigionamento o accumulo senza spingersi troppo in là (dove l'incertezza aumenta).

- **PAST_WINDOW = 168**

- **Cos'è:** il numero di ore di storico passato che il modello usa come input per l'encoder.
- **A cosa serve:** indica quante ore di sequenza storica fornire al modello per prevedere i prossimi 168 passi. Un valore di 168 ore (= 7 giorni) permette di catturare i pattern settimanali e giornalieri: il modello vede un'intera settimana passata per capire ciclicità e trend.
- **Dove viene usato:** in `crea_sequence_dataset(...)`, come lunghezza della "finestra" estratta da `df_features_past_*` per costruire `X_enc`: si preleva sempre l'ultimo blocco di 168 ore (o blocchi mobili durante training) per l'encoder.

- **FUTURE_WINDOW = 168**

- **Cos'è:** il numero di ore futuro che si vuole prevedere in un'unica sequenza.
- **A cosa serve:** definisce quanti passi avanti (ore) il modello produce in output per ogni input encoder: in questo caso 7 giorni completi.
- **Dove viene usato:** sia in `crea_sequence_dataset(...)` per costruire `y` (target di lunghezza 168) e `X_dec` (feature future di lunghezza 168: meteo futuro), sia nella definizione del modello `seq2seq` (o comunque si assume output di lunghezza `future_window`). Allunga la finestra di previsione a un intervallo operativo rilevante.

Parametri di suddivisione del dataset

Suddividiamo i dati storici in 70% per l'addestramento, 15% per la validazione e 15% per il test (`TEST_RATIO = 0.15`, `VAL_RATIO = 0.15`), ottenendo un buon compromesso tra quantità di dati di training e verifica della generalizzazione.

- **TEST_RATIO = 0.15**

- **Cos'è:** la percentuale del dataset storico dedicata al test finale.
- **A cosa serve:** riserva il 15% dei dati storici più recenti per valutare la capacità del modello di generalizzare su dati "mai visti" durante l'addestramento/tuning.
- **Dove viene usato:** in `split_time_series(df_hist, train_ratio, val_ratio)`, per calcolare `n_train`, `n_val`, `n_test`: 15% finale è destinato a test. Garantisce una stima realistica delle performance su dati futuri rispetto allo storico.

- **VAL_RATIO = 0.15**

- **Cos'è:** la percentuale del dataset storico dedicata alla validazione durante l'addestramento.
- **A cosa serve:** riserva il 15% dei dati successivi al training per monitorare l'overfitting e regolare iperparametri, attivare early stopping.
- **Dove viene usato:** in `split_time_series`, e internamente anche in `model.fit(..., validation_split=0.15)` se si usa split su train per validazione. Aiuta a controllare la qualità del modello durante il training.

- **TRAIN_RATIO = 1 - TEST_RATIO - VAL_RATIO**

- **Cos'è:** la quota rimanente (70%) del dataset per il training.
- **A cosa serve:** definisce quanti dati storici usare per addestrare effettivamente il modello, dopo aver riservato validation e test.
- **Dove viene usato:** in `split_time_series`, calcolato automaticamente per garantire che `train + val + test = 100%` dei dati. Scegliere 70/15/15 è una pratica comune per bilanciare quantità di dati di addestramento e di verifica.

Parametri fisici per produzione FV

Simuliamo un impianto da 5 kW di potenza di picco con efficienza 18% (`PANEL_CAPACITY = 5.0`, `EFFICIENCY = 0.18`), valori tipici per un impianto residenziale moderno. Questi parametri, insieme ai dati di irradianze simulati, determinano la scala della produzione oraria.

- **PANEL_CAPACITY = 5.0**

- **Cos'è:** la potenza di picco dell'impianto fotovoltaico, in kW.
- **A cosa serve:** **moltiplicata per efficienza e irradianze per stimare la produzione oraria.** Rappresenta dimensione dell'impianto del prosumer (tipico impianto residenziale di ~5 kW).
- **Dove viene usato:** in `genera_produzione_sintetica(index, meteo_df)`, formula `production = PANEL_CAPACITY * EFFICIENCY * irradianze + rumore`. Influenza direttamente scala e magnitudo della produzione simulata: aumentando questo valore si simula un impianto più grande e maggiore produzione.

- **EFFICIENCY = 0.18**

- **Cos'è:** il rendimento complessivo dell'impianto FV, fra pannelli e inverter (unità fra 0 e 1).

- **A cosa serve:** definisce la frazione di energia solare convertita in elettricità: usata con PANEL_CAPACITY e irradiance per calcolare produzione oraria. Valore tipico di impianti moderni.
- **Dove viene usato:** in genera_produzione_sintetica, nella stessa formula. Influisce sulla relazione tra irradiance simulata e output di produzione: un'efficienza più alta aumenta la stima di kWh prodotti.

Parametri di consumo energetico

Il consumo base orario è fissato a 1.5 kWh (BASE_CONSUMPTION), modulato da un pattern giornaliero e da un effetto temperatura (TEMP_EFFECT_FACTOR = 0.05) che simula riscaldamento o condizionamento quando la temperatura esce dalla fascia di comfort. Questi valori servono a generare un profilo di consumo realistico e variabile in base al meteo.

• BASE_CONSUMPTION = 1.5

- **Cos'è:** consumo energetico orario “base” medio di un prosumer (in kWh), indipendente da temperatura o pattern orari.
- **A cosa serve:** punto di partenza per il profilo di consumo simulato: viene modulato da un pattern giornaliero e da effetti di temperatura. Rappresenta il consumo minimo continuo (elettrodomestici sempre attivi, frigo, apparecchi in standby).
- **Dove viene usato:** in genera_consumo_sintetico, come moltiplicatore di daily_pattern: $\text{consumo} = \text{BASE_CONSUMPTION} * \text{daily_pattern} + \text{effetti temperatura} + \text{rumore}$. Definisce la scala tipica dei consumi simulati.

• TEMP_EFFECT_FACTOR = 0.05

- **Cos'è:** fattore empirico che quantifica come varia il consumo in funzione della deviazione di temperatura dalla fascia “comfort” (ad esempio 20°C).
- **A cosa serve:** quando la temperatura simulata è troppo alta (>22°C) o troppo bassa (<18°C), aggiunge un incremento di consumo proporzionale allo scostamento oltre una soglia di 2°C, moltiplicato per questo fattore. Simula l'uso di riscaldamento o condizionamento.
- **Dove viene usato:** in genera_consumo_sintetico, calcolo $\text{temp_effect} = \text{TEMP_EFFECT_FACTOR} * \max(0, |\text{temp} - 20| - 2)$, poi sommato al consumo base. Influenza la variabilità del consumo legata al meteo, introducendo stagionalità e dipendenza da condizioni estreme.

Descrizione del flusso del programma

Il flusso organizza sequenzialmente le fasi fondamentali di uno studio di forecasting:

1. Generazione del dataset storico sintetico

Si invoca `genera_date_range_storico` per ottenere `idx_hist`, quindi `genera_meteo_sintetico`, `genera_consumo_sintetico`, `genera_produzione_sintetica` per creare DataFrame/Series temporali di meteo, consumo e produzione. I dati vengono puliti con `pulisci_e_imputa` per garantirne la coerenza.

2. Creazione di `df_hist`

Si concatenano le serie in un unico DataFrame con indice orario: colonne `temperature`, `irradiance`, `wind_speed`, `consumption`, `production`. L'immagine sotto è u tipico esempio.

	<code>temperature</code>	<code>irradiance</code>	<code>wind_speed</code>	<code>consumption</code>	<code>production</code>
2024-12-22 08:00:00+01:00	29.368270	0.847078	3.811933	1.547900	0.819483
2024-12-22 09:00:00+01:00	23.053792	0.922625	4.673892	1.367088	0.841816
2024-12-22 10:00:00+01:00	23.455223	0.956161	4.005090	1.406859	0.862677
2024-12-22 11:00:00+01:00	18.973638	0.980126	1.762669	1.288799	0.924551
2024-12-22 12:00:00+01:00	15.023488	0.964870	3.426071	1.546909	0.801658

3. Generazione del dataset meteo futuro

Si calcola `idx_future` come 168 ore successive all'ultimo storico, quindi si genera `meteo_future` e si pulisce. Questi dati serviranno da input decoder per il forecast futuro.

4. Suddivisione in train/val/test

Con `split_time_series`, si divide `df_hist` in porzioni contigue: train (70%), validation (15%), test (15%). Si stampa il range temporale di ciascuna parte.

5. Analisi statistica esplorativa

Per ogni porzione (TRAIN, VAL, TEST) si invoca `analisi_statistica` per descrittive, grafici di serie e matrice di correlazione, aiutando a verificare pattern stagionali, relazioni e anomalie.

6. Preparazione delle sequenze per LSTM

- Per produzione: si costruisce `df_features_past_prod` (meteo storico + produzione storica), si estende il meteo passando da storico a futuro, e si chiama `crea_sequence_dataset` per train e test.
- Per consumo: analogamente con `df_features_past_cons` (meteo + consumo).

Si ottengono `X_enc_train`, `X_dec_train`, `y_train` e le analoghe variabili per test.

7. Costruzione e addestramento dei modelli

- Si definisce `model_cons` e `model_prod` con `build_seq2seq_model`, specificando numero di feature.
- Si addestra ciascun modello con `.fit(...)`, utilizzando `EarlyStopping` e `validation_split`. I risultati di training vengono salvati in `history_cons` e `history_prod` per plottare la curva di perdita.

8. Valutazione sui TRAIN e TEST

- Si calcolano predizioni su train set (`y_pred_train`), si confrontano con i target reali con `valuta_metriche` (MAE, RMSE) per avere un'indicazione di errore interno.
- Si calcolano predizioni su test set (`y_pred_test`) e relative metriche, per misurare la generalizzazione.

9. Visualizzazioni di supporto

- Si tracciano le curve di perdita train vs validation per evidenziare overfitting o convergenza.
- Si utilizza `plot_confronto_window` per mostrare, su finestre specifiche, il confronto reale vs predetto.
- Si utilizza `plot_error_distribution` per visualizzare la distribuzione degli errori complessivi.

10. Previsione futura sui prossimi 7 giorni

- Si estrae l'ultima finestra storica come input encoder e si usa `meteo_future` come input decoder.
- Si eseguono le predizioni con `model_cons.predict` e `model_prod.predict`, ottenendo array di lunghezza 168 ore.
- Si costruisce `df_forecast` con `timestamp` e colonne `forecasted_consumption` e `forecasted_production`.
- Verrà visualizzato un report del tipo :

	forecasted_consumption	forecasted_production	
2025-06-22 12:00:00+02:00	1.621953	1.937455	Produzione > Consumo
2025-06-22 13:00:00+02:00	1.501665	1.802323	
2025-06-22 14:00:00+02:00	1.509356	1.829701	
2025-06-22 15:00:00+02:00	1.500238	1.644140	
2025-06-22 16:00:00+02:00	1.588788	1.463875	Produzione < Consumo
...	...	...	
2025-06-29 07:00:00+02:00	1.497490	1.486987	
2025-06-29 08:00:00+02:00	1.563130	1.461862	Produzione > Consumo
2025-06-29 09:00:00+02:00	1.498985	1.570847	
2025-06-29 10:00:00+02:00	1.616635	1.625112	
2025-06-29 11:00:00+02:00	1.431844	1.723259	

In cui si può percepire una sorta di bilancio energetico su base oraria.

timestamp	bilancio etichetta	
2025-06-22 12:00:00+02:00	0.315502	rilascio
2025-06-22 13:00:00+02:00	0.300658	rilascio
2025-06-22 14:00:00+02:00	0.320345	rilascio
2025-06-22 15:00:00+02:00	0.143902	rilascio
2025-06-22 16:00:00+02:00	-0.124913	acquisto
...	...	...
2025-06-29 07:00:00+02:00	-0.010503	acquisto
2025-06-29 08:00:00+02:00	-0.101268	acquisto
2025-06-29 09:00:00+02:00	0.071863	rilascio
2025-06-29 10:00:00+02:00	0.008477	rilascio
2025-06-29 11:00:00+02:00	0.291416	rilascio

11. Calcolo del bilancio energetico e report

- successivamente viene rilasciato un vero bilancio energetico con indicazione su base oraria se acquistare energia per soddisfare il fabbisogno oppure cederla alla rete.
- Si calcola `net_balance` (produzione – consumo) per ogni ora.
- Si definiscono `to_grid` (surplus positivo da cedere alla rete) e `to_buy` (deficit, energia da acquistare).
- Si stampa un estratto del DataFrame e i totali settimanali di energia da acquistare o da immettere in rete.
- Di seguito una figura che riporta su base oraria :
 1. Previsione di Energia Consumata su base oraria
 2. Previsione di Energia Prodotta da FV su base oraria
 3. Energia Immessa in rete (to grid) su base oraria

4. Energia da Acquistare (to buy) su base oraria
5. Totale Energia Immessa in rete (to grid) nei prossimi 7 gg
6. Totale Energia da Acquistare (to buy) nei prossimi 7 gg

Bilancio energetico calcolato. Ecco un riepilogo:

	forecasted_consumption	forecasted_production
timestamp		
2025-06-22 12:00:00+02:00	1.621953	1.937455
2025-06-22 13:00:00+02:00	1.501665	1.802323
2025-06-22 14:00:00+02:00	1.509356	1.829701
2025-06-22 15:00:00+02:00	1.500238	1.644140
2025-06-22 16:00:00+02:00	1.588788	1.463875

	to_grid	to_buy
timestamp		
2025-06-22 12:00:00+02:00	0.315502	0.000000
2025-06-22 13:00:00+02:00	0.300658	0.000000
2025-06-22 14:00:00+02:00	0.320345	0.000000
2025-06-22 15:00:00+02:00	0.143902	0.000000
2025-06-22 16:00:00+02:00	0.000000	0.124913

 Riepilogo energia per i prossimi 7 giorni:

Energia da immettere in rete [kWh]: 11.95

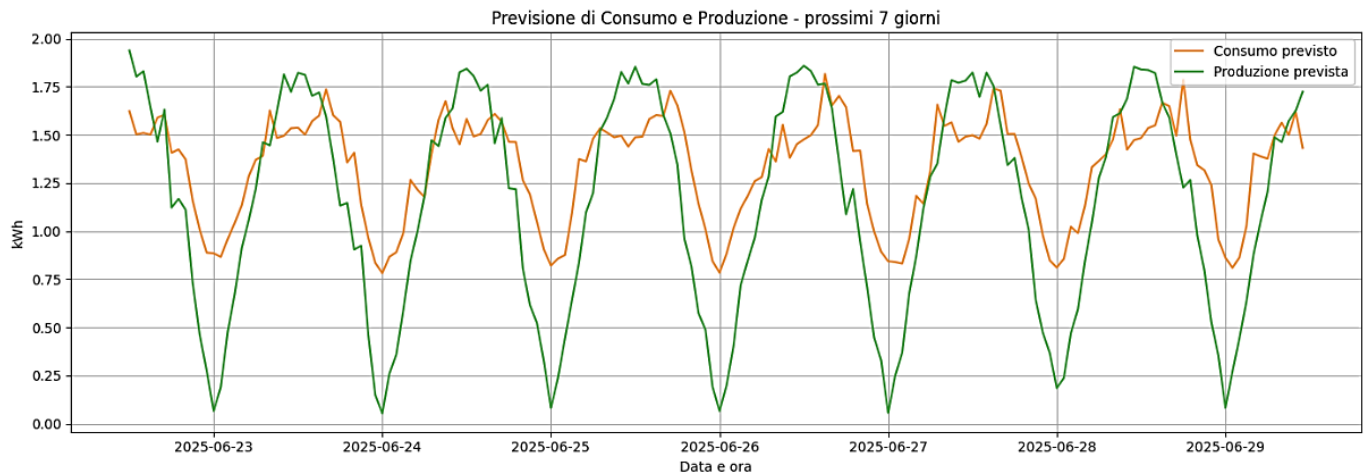
Energia da acquistare [kWh]: 42.13

NB: Questo report è forse il punto principale dell'intera applicazione in quanto permette di operare una oculata programmazione degli acquisti di energia presso il proprio fornitore.

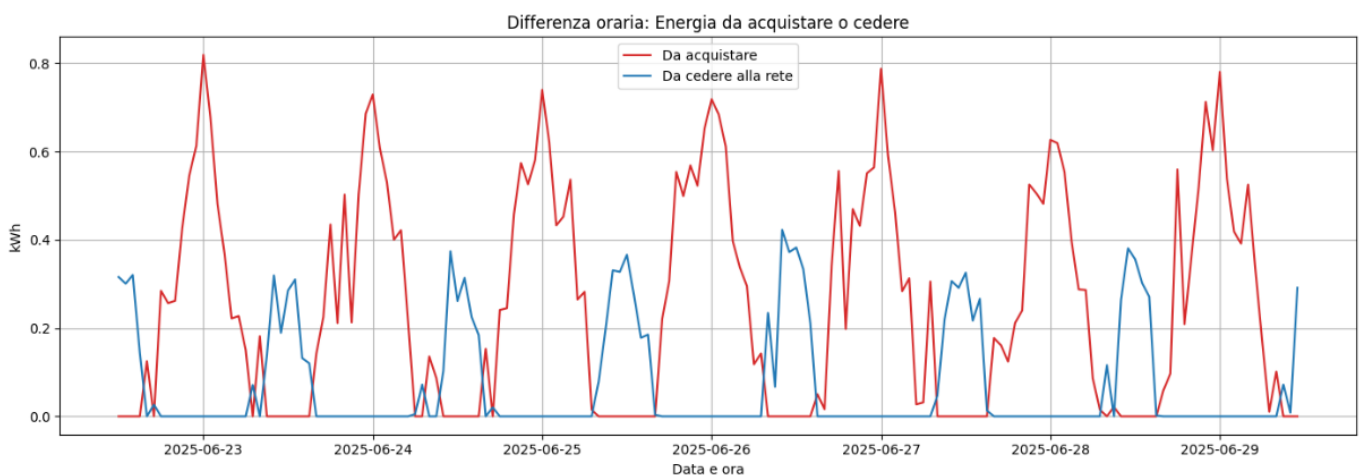
12. Grafici riepilogativi finali

- 1) Serie temporali di consumo e produzione previsti.
- 2) Serie oraria di to_buy e to_grid.
- 3) Grafico cumulativo settimanale dei volumi di energia acquistata o immessa.

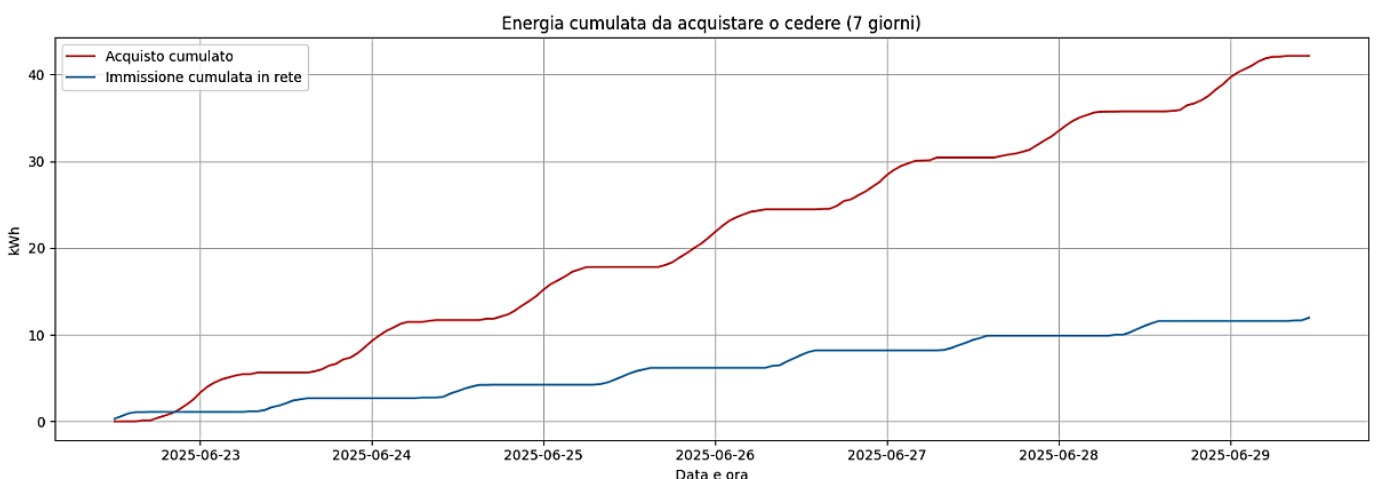
Il primo grafico riporta la previsione dei prossimi 7 giorni del consumo (arancio) e della produzione da fotovoltaico (verde)



Il secondo grafico riporta le previsioni su base 7 giorni della differenza su base oraria tra energia da acquistare (rosso) ed energia da cedere alla rete (azzurro)



Il terzo grafico serve a visualizzare l'andamento cumulativo nel tempo dell'energia da acquistare e di quella immessa in rete, per valutare l'accumulo totale su un periodo di 7 giorni.



Conclusione

Il documento descrive uno studio di forecasting multistep per l'equilibrio energetico di un prosumer, combinando simulazione di dati sintetici e *modelli LSTM seq2seq*. La sezione introduttiva inquadra la rilevanza delle previsioni orarie in un contesto di energia rinnovabile, evidenziando l'impatto operativo di stimare consumi e produzioni future. La descrizione generale espone con linguaggio tecnico ma scorrevole le finalità dell'applicazione, le possibilità di personalizzazione e i vantaggi di un approccio modulare. Le descrizioni sintetiche delle librerie e delle funzioni facilitano la comprensione delle parti costitutive, mentre il riepilogo del flusso mostra come le fasi di generazione dati, preprocessing, modellazione, valutazione e reporting si concatenino in un processo coerente. In conclusione, questa documentazione funge sia da guida allo studio metodologico sia da manuale d'uso, permettendo all'utente di integrare immagini, grafici e commenti aggiuntivi per approfondire ogni aspetto, e di adattare il prototipo a scenari reali o a esigenze specifiche. Buon proseguimento nell'analisi !